K. N. Toosi University of Technology

Faculty of Electrical Engineering

System Identification Project Report

Instructor: Dr. Aliyari

Authors: Hossein Mosaffa, Ali Sheikholeslami

February 2020 (Bahman 1398)

Abstract

This project applies the Least Squares Iterative (LSI) idea to system identification. Ordinary least squares (LS) fails when the system's internal variables are unavailable, or when the regressor vector contains noise terms that cannot be measured. LSI handles this by replacing the unknown terms with their estimates and refining them over several iterations.

Paper 1 applies LSI to a multivariable controlled ARMA (CARMA) system. Paper 2 applies it to a Box-Jenkins model. Paper 3 applies it to a Wiener nonlinear model. The simulations for each paper were implemented and are reported below.

Keywords: LSI, CARMA, Box-Jenkins, Wiener model

Repository Layout

- codes/E1/CARMA_lsi.m: Paper 1 (CARMA).
- codes/E2/BOXJENKINS_ITERATIVE.m and BOXJENKINS_RECURSIVE.m: Paper 2 (Box-Jenkins).
- codes/E3/lsi1.m, lsi2.m, lsi3.m: Paper 3 (Wiener model).

Note: the original Persian report includes many MATLAB plots (parameter estimates per iteration, error curves, flowcharts). They are not reproduced in this translation. Add them from the original report or regenerate them with the scripts.

Paper 1: LSI for a Multivariable CARMA System

Reference: Bao et al., "Least squares based iterative parameter estimation algorithm for multivariable controlled ARMA system modelling with finite measurement data" [1].

Model

ARMA stands for AutoRegressive Moving Average. The system is a multivariable controlled ARMA process. The main difficulty is the presence of random noise, so estimation becomes an error-minimization (optimization) problem: the estimate of the true parameters always has some error, and we want to minimize it.

```
A(z) y(t) = B(z) u(t) + D(z) v(t)
```

Here u(t), y(t) and v(t) are the input, output and noise, with t = 1, 2, ..., L, where L is the data length. A(z), B(z) and D(z) are matrix polynomials in the unit delay z^-1:

```
A(z) = I + A1 z^-1 + ... + A_na z^-na
B(z) = B1 z^-1 + ... + B_nb z^-nb
D(z) = I + D1 z^-1 + ... + D_nd z^-nd
```

The matrices Ai, Bi and Di have sizes m x m, m x r and m x m, where m is the output dimension and r is the input dimension. The orders are assumed known, and u, y and v are zero for t <= 0. The key issue with plain LS is that the information vector contains unmeasurable noise terms. The iterative algorithm estimates these terms and uses the estimates.

Least squares formulation

Expanding the model gives:

```
y(t) = -A1 y(t-1) - ... - A_na y(t-na) + B1 u(t-1) + ... + B_nb u(t-nb) + D1 v(t-1) + ... + D_nd v(t-nd) + v(t)
```

Written in vector-matrix form:

```
y(t) = theta^T phi(t) + v(t)
theta^T = [A1, ..., A_na, B1, ..., B_nb, D1, ..., D_nd]   (m x n, n = m*na + r*nb + m*nd)
phi(t) = [-y(t-1); ...; -y(t-na); u(t-1); ...; u(t-nb); v(t-1); ...; v(t-nd)]   (n x 1)
```

Stacking all observations gives Y_L = theta^T Phi_L + V_L, with Y_L (m x L), Phi_L (n x L) and V_L (m x L). The cost function is J = ||Y_L - theta^T Phi_L||^2. Setting its derivative to zero gives:

```
dJ/dtheta = -2 (Y_L - theta^T Phi_L) Phi_L^T = 0
theta = (Phi_L Phi_L^T)^-1 Phi_L Y_L^T
```

This is not usable directly because Phi_L contains the unmeasurable noise terms v(t-i). So the unknown terms are replaced by their estimates, starting from random initial values, and the estimate is refined until it converges within a tolerance.

Algorithm

1. Collect input and output data for t = 1..L. The input is a zero-mean, unit-variance sequence. Build the stacked output matrix Y_L = [y(1), y(2), ..., y(L)].
2. Set the iteration counter k = 1 and initialize the noise estimates v0(t) with random vectors for t = 1..L.
3. Build the information vector estimates phi_k(t) for each t, using v_{k-1}(t-i) in place of the unknown noise terms. Stack them into Phi_k(L).
4. Update the parameter estimate: theta_k = (Phi_k Phi_k^T)^-1 Phi_k Y^T.
5. Update the noise estimate: v_k(t) = y(t) - theta_k^T phi_k(t).

6. Compare ||theta_k - theta_(k-1)|| with a tolerance. If it is smaller, stop and take theta_k as the final estimate. Otherwise increase k and repeat. The estimation error is error(%) = || theta_k - true theta|| / ||true theta|| x 100.

Simulation

The paper's example is a two-input, two-output system:

$$y(t) + A1\ y(t-1) = B1\ u(t-1) + v(t) + D1\ v(t-1)$$
$$A1 = [0.6\ 0.5;\ -0.8\ 1],\ \ B1 = [1.5\ -0.4;\ -0.5\ 1.1],\ \ D1 = [0.2\ -0.1;\ -0.1\ 0.6]$$

The inputs u1 and u2 are zero-mean, unit-variance sequences. The noise is generated the same way with variance 0.25. The algorithm was implemented in MATLAB with a for loop and a stop condition, for L = 1000, 2000 and 3000.

True parameter values: a11 = 0.6, a12 = 0.5, b11 = 1.5, b12 = -0.4, d11 = 0.2, d12 = -0.1, a21 = -0.8, a22 = 1, b21 = -0.5, b22 = 1.1, d21 = -0.1, d22 = 0.6.

Results

In each case the algorithm reached the optimal estimate in 9 iterations. Estimates at the first and last iteration:

Param	**True**	**L=1000 k=1**	**L=1000 k=9**	**L=2000 k=1**	**L=2000 k=9**	**L=3000 k=1**	**L=3000 k=9**
a11	0.6	0.6002	0.6002	0.6	0.6	0.6	0.6
a12	0.5	0.4998	0.4998	0.5	0.5	0.5	0.5
b11	1.5	1.5004	1.5011	1.5089	1.5057	1.5056	1.5065
b12	-0.4	-0.4037	-0.4031	-0.4057	-0.4073	-0.3996	-0.4016
d11	0.2	0.0101	0.1877	0.0079	0.1918	0.0128	0.2063
d12	-0.1	-0.0146	-0.0928	0.0023	-0.0811	-0.0036	-0.0968
a21	-0.8	-0.8001	-0.8001	-0.7999	-0.8	-0.8	-0.8
a22	1	0.9999	0.9999	1	1	1	1
b21	-0.5	-0.4956	-0.501	-0.5078	-0.4984	-0.4922	-0.494
b22	1.1	1.0925	1.0849	1.1069	1.1095	1.0957	1.1005
d21	-0.1	-0.0009	-0.0889	0.0284	-0.1067	0.002	-0.1063
d22	0.6	-0.0001	0.5612	-0.0106	0.619	-0.0134	0.5736
Error %		29.11	2.0668	30.0808	1.374	29.8357	1.3129

Error (%) per iteration:

k	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
L=1000	29.11	9.1235	4.7676	3.1561	2.4856	2.2078	2.1329	2.0846	2.0668
L=2000	30.0808	7.636	2.692	1.3544	1.4268	1.2944	1.3088	1.3716	1.374

L=3000	29.8357	9.1126	3.7211	2.1756	1.706	1.4878	1.355	1.3332	1.3129

Discussion

Near convergence the error may oscillate slightly, but overall it falls sharply as k increases. A longer data set generally gives a lower error, probably because more data improves estimation accuracy. The effect is clearest in the early iterations and near convergence. Across repeated runs the stop condition was met after 8, 9 or 10 iterations, so convergence is fast and the method is a strong tool for parameter estimation.

Paper 2: LSI for Box-Jenkins Models

Reference: Liu, Wang and Ding, "Least squares based iterative algorithms for identifying Box-Jenkins models with finite measurement data" [2].

Model

Box-Jenkins models can be identified from finite data with both recursive and iterative algorithms. Unmeasurable variables, such as the noise dynamics inside the information vector, are replaced by estimates with fixed initial conditions. Iterative algorithms converge faster and are more accurate than online recursive ones. The system here is single-input, single-output:

```
y(t) = [B(z)/A(z)] u(t) + [D(z)/C(z)] v(t)
```

u(t) is a zero-mean, unit-variance persistently exciting input, and v(t) is zero-mean white noise. A, B, C and D are scalar polynomials in z^-1 with orders na = 2, nb = 2, nc = 1, nd = 1:

```
A(z) = 1 + a1 z^-1 + a2 z^-2
B(z) = b1 z^-1 + b2 z^-2
C(z) = 1 + c1 z^-1
D(z) = 1 + d1 z^-1
```

True parameters: theta = [a1, a2, b1, b2, c1, d1] = [-0.4, 0.44, -1.8, 0.24, -0.48, 0.85].

Recursive algorithm (BJ-RGELS)

1. Collect input and output data for t = 1..L, with the input zero for t <= 0.
2. Set the initial conditions: p0 = 10^6, theta0 = 1/p0 (all entries), P0 = p0 I, and x(i) = w(i) = v(i) = 1/p0 for i <= 0.
3. Update the information vectors at each step t: phi_s(t) = [-x(t-1); -x(t-2); u(t-1); u(t-2)], phi_n(t) = [-w(t-1); v(t-1)], phi(t) = [phi_s(t); phi_n(t)]. These have sizes 4x1, 2x1 and 6x1.
4. Update the covariance: P(t) = P(t-1) - P(t-1) phi(t) phi(t)^T P(t-1) / (1 + phi(t)^T P(t-1) phi(t)).
5. Update the parameters: theta(t) = theta(t-1) + P(t) phi(t) [y(t) - phi(t)^T theta(t-1)].
6. Update the estimates: x(t) = phi_s(t)^T theta_s(t) (noise-free output), w(t) = y(t) - x(t) (colored noise), v(t) = w(t) - phi_n(t)^T theta_n(t) (white noise).

7. Advance t until all L samples are processed, giving theta(L).

Recursive results for noise variance 0.25 and 1, with L = 1000 and 2000:

Noise variance	L	a1	a2	b1	b2	c1	d1	Error %
0.25	1000	-0.3592	0.426	-1.8016	0.1726	-0.3954	0.8924	5.7753
0.25	2000	-0.3741	0.428	-1.7995	0.1889	-0.4282	0.8844	3.9798
1	1000	-0.5014	0.4817	-1.7846	0.4396	-0.54	0.8027	11.2152
1	2000	-0.4569	0.3966	-1.8245	0.3117	-0.4454	0.85	5.1191
True		-0.4	0.44	-1.8	0.24	-0.48	0.85	

More data gives a better estimate, although this holds in general and not on every single run, since the noise is random. Higher noise variance noticeably degrades the estimate.

Iterative algorithm

The iterative algorithm needs a finite data length, so the analysis is offline (batch).

1. Collect input and output data for t = 1..L, with u(t) = 0 for t <= 0, and form the stacked output Y_L = [y(L); y(L-1); ...; y(1)], an L x 1 vector.
2. Set k = 1 and initialize x0(t), w0(t), v0(t) with random values.
3. Update for all t: phi_s,k(t) = [-x_{k-1}(t-1); ...; -x_{k-1}(t-na); u(t-1); ...; u(t-nb)], phi_n,k(t) = [-w_{k-1}(t-1); ...; -w_{k-1}(t-nc); v_{k-1}(t-1); ...; v_{k-1}(t-nd)], phi_k(t) = [phi_s,k(t); phi_n,k(t)]. Then stack them into Phi_k(L).
4. Update the parameter estimate: theta_k = (Phi_k^T Phi_k)^-1 Phi_k^T Y_L.
5. Update x_k(t) = phi_s,k(t)^T theta_s,k, w_k(t) = y(t) - x_k(t), and v_k(t) = y(t) - phi_k(t)^T theta_k.
6. Stop if ||theta_k - theta_(k-1)|| is below a tolerance. Otherwise continue.

The reason for iterating is the same as in Paper 1. The LS cost is J(theta) = ||Y - Phi theta||^2 with solution theta = (Phi^T Phi)^-1 Phi^T Y. But Phi contains unknown noise-free outputs, colored noise and white noise terms, so this estimate is poor. The iterative scheme substitutes their estimates and updates them each pass.

In the experiments the iteration limit was first set to 100, and the algorithm converged and stopped around iteration 18 to 22. Limits of 50 and 25 behaved the same way. The paper uses k = 10, and for all cases here the iteration count was fixed at 15.

Results: L = 1000

Noise variance 0.25 (true theta = [-0.4, 0.44, -1.8, 0.24, -0.48, 0.85]):

k	a1	a2	b1	b2	c1	d1	Error %
1	-0.0144	0.0248	-1.8103	-0.5167	0.1584	0.1801	61.6784

2	-0.9388	0.6043	-1.7857	1.2141	-0.679	-0.4051	79.1073
3	-0.4194	0.4427	-1.8034	0.2752	-0.1977	0.4998	21.0489
4	-0.4026	0.4434	-1.8045	0.2483	-0.3698	0.6456	10.8316
5	-0.403	0.4463	-1.8032	0.248	-0.4713	0.6713	8.3551
6	-0.4019	0.4455	-1.8068	0.2462	-0.4681	0.7248	5.881
7	-0.4012	0.4448	-1.8032	0.2443	-0.4726	0.757	4.3613
8	-0.4009	0.4453	-1.8037	0.2446	-0.4683	0.7818	3.2474
9	-0.4016	0.4454	-1.8035	0.2464	-0.4646	0.8035	2.3214
10	-0.4015	0.4456	-1.8034	0.2458	-0.4693	0.8063	2.1355
11	-0.4018	0.4458	-1.8037	0.2459	-0.4651	0.8163	1.7691
12	-0.4021	0.4456	-1.8026	0.2469	-0.4638	0.8267	1.3929
13	-0.4018	0.4458	-1.8031	0.2463	-0.4646	0.8306	1.2333
14	-0.4019	0.4458	-1.8022	0.2465	-0.4662	0.8296	1.2261
15	-0.402	0.446	-1.8019	0.2467	-0.4642	0.8332	1.1606

Noise variance 1:

k	**a1**	**a2**	**b1**	**b2**	**c1**	**d1**	**Error %**
1	0.1076	0.1766	-1.8327	-0.5367	-0.1738	0.1975	56.139
2	0.4537	0.1732	-1.7878	-1.3454	0.0214	0.7073	88.245
3	-0.3772	0.4262	-1.7938	0.1802	-0.4391	0.6002	12.187
4	-0.3838	0.428	-1.7938	0.1997	-0.4964	0.6925	7.6795
5	-0.4059	0.4411	-1.8042	0.2455	-0.4966	0.7608	4.2483
6	-0.3978	0.4314	-1.8114	0.2317	-0.5017	0.7933	2.9338
7	-0.3915	0.4311	-1.8181	0.2199	-0.5	0.8206	2.1576
8	-0.3951	0.4318	-1.816	0.2252	-0.5011	0.8341	1.6571
9	-0.3956	0.4314	-1.8144	0.2262	-0.5026	0.8348	1.6384
10	-0.3911	0.4301	-1.8105	0.2176	-0.5034	0.8339	1.8631
11	-0.3956	0.4331	-1.816	0.2265	-0.5038	0.8295	1.8005
12	-0.3965	0.4314	-1.8131	0.228	-0.499	0.841	1.3541
13	-0.393	0.4312	-1.8141	0.2215	-0.5034	0.8409	1.6792
14	-0.3949	0.4318	-1.815	0.2252	-0.4998	0.8483	1.4237
15	-0.3953	0.4316	-1.8145	0.2254	-0.5009	0.8522	1.4426

With the higher noise variance the final error is larger. For the same data length and noise variance, the iterative algorithm gives a much lower error than the recursive one, and its error trends downward over the iterations.

Results: L = 2000

Noise variance 0.25:

k	a1	a2	b1	b2	c1	d1	Error %
1	-0.08	-0.0939	-1.7373	-0.4443	0.0425	-0.0772	65.7823
2	-0.764	0.5308	-1.7807	0.8661	-0.0072	0.3087	47.7479
3	-0.4033	0.4291	-1.7911	0.2482	-0.0986	0.6465	20.164
4	-0.3935	0.4391	-1.7891	0.2305	-0.3849	0.6609	9.8917
5	-0.3965	0.4413	-1.7911	0.2369	-0.4641	0.6809	7.929
6	-0.3948	0.44	-1.7923	0.2346	-0.4678	0.7309	5.6016
7	-0.3941	0.4403	-1.7945	0.2336	-0.4697	0.773	3.6502
8	-0.3943	0.4403	-1.7938	0.2326	-0.4727	0.7944	2.6653
9	-0.3945	0.44	-1.7941	0.2336	-0.4707	0.8126	1.8609
10	-0.3945	0.4402	-1.7945	0.2333	-0.4691	0.8262	1.3081
11	-0.3944	0.4399	-1.7941	0.2333	-0.4719	0.8291	1.155
12	-0.3943	0.4401	-1.7941	0.233	-0.4688	0.8382	0.9101
13	-0.3946	0.4402	-1.7936	0.2336	-0.4709	0.8382	0.8512
14	-0.3944	0.4402	-1.7937	0.2332	-0.4691	0.8438	0.7719
15	-0.3946	0.4402	-1.7933	0.2337	-0.4706	0.8442	0.7133

Noise variance 1:

k	a1	a2	b1	b2	c1	d1	Error %
1	0.1582	0.1124	-1.805	-0.4939	0.0219	0.179	60.0407
2	0.4386	0.1977	-1.8032	-1.2677	0.953	1.6221	111.1179
3	-0.3635	0.428	-1.8057	0.1861	-0.4267	0.5689	13.6898
4	-0.3733	0.4357	-1.8007	0.2078	-0.4664	0.6774	8.3041
5	-0.3935	0.4487	-1.8036	0.2438	-0.4729	0.7356	5.3703
6	-0.3976	0.444	-1.7985	0.2507	-0.4691	0.7802	3.3394
7	-0.393	0.4443	-1.7985	0.2449	-0.4741	0.7989	2.4399
8	-0.3914	0.4454	-1.7948	0.2413	-0.4715	0.8199	1.5531
9	-0.3928	0.4464	-1.7962	0.2446	-0.4707	0.8398	0.8315
10	-0.3929	0.4458	-1.7937	0.2453	-0.4727	0.8492	0.6766
11	-0.394	0.4472	-1.7952	0.2474	-0.4711	0.8577	0.8121
12	-0.394	0.4464	-1.7938	0.2471	-0.4728	0.8595	0.8183
13	-0.3941	0.4469	-1.7963	0.2474	-0.4717	0.8629	0.9174
14	-0.3946	0.4464	-1.7959	0.2481	-0.472	0.8652	0.9885
15	-0.3941	0.4463	-1.7975	0.2473	-0.4723	0.8656	0.9728

Discussion

The iterative error is lower than the recursive error in every case, and it generally decreases as the iteration count and the data length grow. Comparing the error curves for variance 0.25 and 1 at L = 2000 shows that higher noise variance raises the overall estimation error. The iterative approach converges faster and estimates more accurately than the recursive one. Because noise is random, these conclusions hold in general over many runs, and individual runs can deviate slightly.

Paper 3: LSI for Wiener Nonlinear Systems

Reference: Zhou, Li and Pan, "Least-squares-based iterative identification algorithm for Wiener nonlinear systems" [3].

Model

This paper identifies nonlinear dynamic systems with a Wiener model and estimates the parameters with LSI. A Wiener model is a linear dynamic block followed by a static nonlinear block. In the paper the linear block is x(t) = [B(z)/A(z)] u(t), and the nonlinear block is a polynomial in x(t) with coefficients gamma_1, gamma_2, ... , with y(t) the (noisy) output. Substituting x(t) into the nonlinear block directly would make the equation complicated, so the paper rewrites the relation between u(t) and x(t) and assumes gamma_1 = 1. The input-output relation then becomes linear in the parameters a, b and gamma, and with suitable regressors we can write y(t) = phi^T(t) theta + noise, where theta = [a1, a2, b1, b2, gamma_2, gamma_1-related terms]. (The original equations were images in the source report; see the cited paper for the exact expressions.)

LSI algorithm

The problem is that x(t) is not measurable. So at each iteration the regressor phi_k(t) is built using the estimate of x(t) from the previous iteration, and theta_k is computed by least squares. Then x_k(t) is recomputed from the new parameters, and the loop repeats. The algorithm stops when theta changes by less than a small epsilon between iterations. A flowchart of LSI is included in the original report.

Simulation

The simulation code for the paper's example is in weiner_lsi.m. The procedure is:

1. Define the input data as random noise, a reasonable choice for simulation since it excites all frequencies.
2. Assume zero initial conditions for x and compute the system output.
3. Add noise with standard deviation 0.1, and separately 0.2, to the output to get y.
4. Build Y as in the formulas, compute phi using x from the previous step, and compute theta.

The initial condition for x causes a problem. If x starts at zero or any constant, the matrix to be inverted for theta becomes singular (determinant zero). There are two fixes. One is to use random initial conditions. The other is to keep zero initial conditions but use a regularized estimate with a suitable mu:

```
theta = (Phi_k^T Phi_k + mu_k I_n0)^-1 Phi_k^T Y
```

The delta metric tracked at each step is delta = ||theta_k - theta|| / ||theta||, where theta is the true parameter vector.

Results

Parameter	True	No noise	Noise std 0.1	Noise std 0.2
A1	-0.18	-0.18	-0.1821	-0.1786
A2	0.44	0.44	0.4405	0.441
B1	0.58	0.5799	0.5785	0.577
B2	0.41	0.41	0.4078	0.407
gamma1	-0.45	-0.4501	-0.4503	-0.4602
gamma2	0.25	0.2501	0.2513	0.256

With separate training and test data, the fit was 92% for noise std 0.1 and 86.25% for noise std 0.2. The delta plots and test-error plots for each case are in the original report.

The method was also tried on datasets from DaISy (heating system and heat exchanger). It did not give a good fit there. The code is in weiner_lsi3.m.

References

1. Bao, Bo, et al. "Least squares based iterative parameter estimation algorithm for multivariable controlled ARMA system modelling with finite measurement data." Mathematical and Computer Modelling 53.9-10 (2011): 1664-1669.
2. Liu, Yanjun, Dongqing Wang, and Feng Ding. "Least squares based iterative algorithms for identifying Box-Jenkins models with finite measurement data." Digital Signal Processing 20.5 (2010): 1458-1467.
3. Zhou, Lincheng, Xiangli Li, and Feng Pan. "Least-squares-based iterative identification algorithm for Wiener nonlinear systems." Journal of Applied Mathematics 2013 (2013).